Architectural Isolation as a Timing Safety Primitive for Edge AI Medical Devices: Controlled Experimental Evidence on a Shared-Silicon Platform

Akul Mallayya Swami

***Abstract*—A system can satisfy accuracy-based validation, maintain output stability (Safety-Threshold Exceedance Rate, STER, equal to zero), and still violate timing constraints under deployment load. These are structurally independent properties that current pre-market validation protocols often do not operationalize at the inference layer. This letter demonstrates their independence through a controlled same-hardware experiment: identical MobileNetV2 models are evaluated under identical adversarial load on two execution paths of the same NVIDIA Jetson Orin Nano Super, a dedicated graphics processing unit (GPU) accelerator (TensorRT FP16, half-precision floating point) and a general-purpose central processing unit (CPU) (Open Neural Network Exchange (ONNX) Runtime FP32, single-precision floating point). Both paths maintain STER$= 0$; the CPU path (ONNX Runtime FP32) degrades 7.2$\times$ under combined load (mean latency 9.8$\times$ higher than the GPU path (TensorRT FP16), which maintains latency below 11 ms), breaching the 10 Hz clinical cycle budget by 65%. Joint STER and latency verification is proposed as a candidate method for operationalizing U.S. Food and Drug Administration (FDA) Draft Guidance FDA-2024-D-4488 robustness requirements at the inference layer, subject to regulatory review and clinical validation.**

***Index Terms*—edge artificial intelligence (AI), inference stability, timing safety, architectural isolation, Safety-Threshold Exceedance Rate (STER), Software as a Medical Device (SaMD), pre-market validation, IEC 62304, ISO 14971, TensorRT, ONNX Runtime.**

I. Introduction

FDA pre-market validation of AI-enabled medical devices evaluates model accuracy (sensitivity, specificity, area under the receiver operating curve (AUC)) under controlled, zero-contention conditions [1]. This framework does not characterize whether inference outputs remain within bounds, or whether inference timing is preserved within clinical cycle budgets, under realistic deployment load. A 2025 cross-sectional study of 950 FDA-cleared AI devices found diagnostic errors were the leading recall cause, with 43% occurring within the first year [14]. International Electrotechnical Commission (IEC) 62304 Clause 6.3.2 requires software timing verification under foreseeable conditions [8], yet accuracy-based certification cannot detect timing violations under deployment load.

This letter demonstrates a challenge in operationalizing Software as a Medical Device (SaMD) robustness requirements: while FDA-2024-D-4488 requires assessment under foreseeable conditions of use [1], accuracy-based certification protocols do not jointly verify output stability and timing reliability at the inference layer under deployment load. The safety criterion is not latency magnitude but whether latency satisfies the clinical deadline; a change in accuracy (Δacc) of less than 1.0% does not indicate deadline compliance. The experiment holds all hardware variables constant so any divergence arises from execution context alone. Results show STER$= 0$ on both paths while CPU P_{99} (99th-percentile latency) exceeds the 10 Hz budget by 65% under combined load. A joint STER and latency protocol, consistent with FDA-2024-D-4488, is described.

Manuscript received April 2026. This work was conducted independently.

A. M. Swami is with Varian Medical Systems, A Siemens Healthineers Company, Palo Alto, CA 94304 USA (e-mail: swami.akul@alumni.uml.edu). ORCID: 0009-0003-9549-5543. This work was conducted independently and does not represent the views of Varian Medical Systems or Siemens Healthineers. Portions of this manuscript were drafted with the assistance of Claude (Anthropic); the author reviewed, verified, and takes full responsibility for all content, experimental data, and claims.

II. Related Work

Prior edge inference work addresses throughput maximization [2], latency predictability under GPU partitioning [3], and cross-device accuracy benchmarking [4], [5], measuring latency or accuracy, not their independence under shared-silicon contention. Safety-oriented work on neural networks in medical devices [6] addresses model-level mechanisms but does not empirically characterize resource-contention-induced timing failure as a distinct property from output correctness. Established methods such as worst-case execution time (WCET) analysis [7] address timing at the scheduler level but do not characterize inference output stability under deployment-induced contention. No prior work has used a same-hardware controlled design to empirically demonstrate that output stability and timing reliability are orthogonal properties, or proposed their joint measurement as a pre-market acceptance condition for SaMD.

III. System Model and Formal Definitions

A. Output Stability Under Identical Inputs

Let f denote a deployed inference model and $\mathbf{x}_i$ a fixed test input. Under stressor condition c, the per-inference output deviation is:

$$\delta_i = \left\|\sigma(\mathbf{y}_i^c) - \bar{\sigma}_i\right\|_\infty = \max_k \left|\sigma(\mathbf{y}_i^c)_k - \bar{\sigma}_{i,k}\right| \tag{1}$$

where $\sigma(\mathbf{y}_i^c)$ is the softmax output under condition c and $\bar{\sigma}_i$ is the per-image reference at zero load.

B. STER Definition

Definition 1 (T^*): $T^* = 0.05$, chosen to detect subthreshold distribution shifts below typical clinical tolerance bands. Clinical validation of this threshold for specific device classes is required [9].

Definition 2 (STER): For N inference activations:

$$\text{STER} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\delta_i > T^*] \tag{2}$$

Accuracy testing evaluates $\text{argmax}(\sigma(\mathbf{y}))$ and cannot detect subthreshold distribution drift; a model shifting confidence by 4.99% passes accuracy testing. STER detects this class of silent deviations. STER is a candidate pre-market metric, not an existing FDA-specified standard.

C. Robustness Confirmation Condition

The joint pre-market acceptance condition is:

$$\text{STER}_c \leq \text{STER}_{\max} \quad \text{AND} \quad \text{latency}_{P_{99}} \leq T_n \tag{3}$$

where $\text{STER}_{\max} = 0$, $\text{latency}_{P_{99}}$ is the 99th-percentile inference latency, and $T_n = 100$ ms is the nominal cycle period at 10 Hz (a representative constraint; appropriate rates are device-specific). STER and latency are orthogonal safety dimensions; both must be verified.

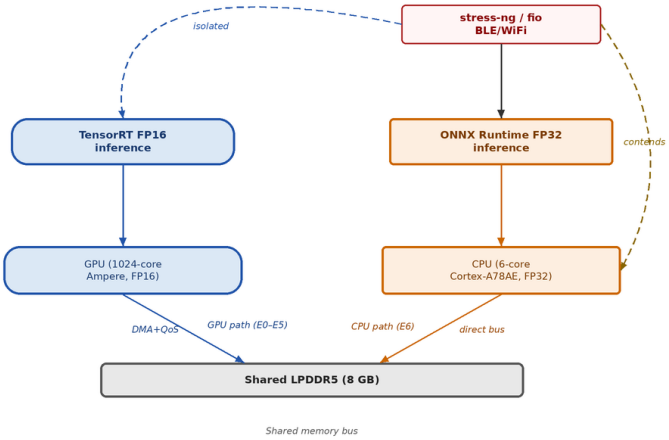

Fig. 1. Jetson Orin Nano Super execution paths. GPU path (TensorRT FP16) uses dedicated DMA channels with QoS bandwidth reservation. CPU path (ONNX Runtime FP32) competes directly for bus bandwidth with stressor workloads.

D. Architectural Model

The Jetson Orin Nano Super uses shared Low-Power Double Data Rate 5 (LPDDR5) memory; CPU, GPU, and memory controller compete for the same bus (Fig. 1). The GPU path (TensorRT FP16) offloads inference to the 1024-core Ampere GPU via dedicated Direct Memory Access (DMA) paths with Quality of Service (QoS) bandwidth reservation. The CPU path (ONNX Runtime FP32) runs on the 6-core Arm Cortex-A78AE, competing directly with stressor workloads for CPU time, cache, and dynamic random-access memory (DRAM) bandwidth.

IV. Hardware Platform and Experimental Protocol

A. Hardware Platform

Both execution paths run MobileNetV2 [11] (224×224 red-green-blue (RGB), ImageNet 1000-class head) on the same 500-image fixed test set. The GPU path runs TensorRT FP16 [12] at a nominal 10 Hz (T_n = 100 ms); the CPU path runs MobileNetV2 FP32 via ONNX Runtime [13] on the same hardware. Same-device testing eliminates chip-to-chip variation and thermal confounds [3]. No thermal throttling events were recorded (`tegrastats` monitoring throughout).

TABLE I. Hardware Platform Summary

Component	Role	Key Spec.
Jetson Orin Nano Super (GPU)	Inference host	TensorRT FP16
Jetson Orin Nano Super (CPU)	Same hardware	ONNX RT FP32
nRF52840 DK + TI SensorTag	BLE stressor	BLE 5.0, 6 conn.
GL-AX1800 WiFi 6 Router	RF environment	2.4 GHz

B. Experimental Protocol

Each trial: $N = 500$ inference activations; softmax $\sigma(\mathbf{y}_i)$ and latency (via `time.perf_counter()`) recorded per activation; δ_i computed per (1); STER computed per (2). All CPU inferences used `CPUExecutionProvider` in deterministic mode. Trial counts ($N = 500$; 10 trials per stressor level) provide stable mean and P_{99} estimates. The 9.8× mean latency difference (10.6 ms GPU vs. 104.0 ms CPU) is large relative to within-condition variability (CPU SD = 5.7 ms under combined load), indicating a robust effect; confidence intervals and formal hypothesis tests are deferred to a full-length follow-up study.

TABLE II. Experiment Summary

ID	Stressor	Protocol	IEC[a]
E0	Baseline	Both paths, zero load. Ref. $\bar{\sigma}_i$ captured. 10×500 inf.	B+C
E1	CPU stress	`stress-ng` [10] 25/50/75/100%. Both paths. 10×500 inf./level.	B+C
E2–E4	Isolation	Mem. (25–90%), GPU co-tenancy (0–4), net I/O (BLE 0–6+WiFi). GPU only; STER= 0.	B+C
E5	Combined	CPU 75%, mem. 50%, BLE 4+fio. Both paths. 10×500 inf.	B+C
E6	CPU char.	CPU path only. E0+E1+E5 conditions. 10×500 inf.	B/C

[a] IEC 62304: B=Cl. 5.7 (integration); C=Cl. 5.8 (system testing).

TABLE III. E0 Baseline. δ_{mean}: Mean output deviation (1); Lat. SD: per-trial SD; P_{99}: 99th-pct. latency.

Path	STER	δ_{mean}	Lat. mean	Lat. SD	P_{99}
GPU (TRT FP16)[†]	0.0000	0.0182	10.6 ms	0.2 ms	10.9 ms
CPU (ONNX FP32)	0.0000	0.0000	14.5 ms	0.2 ms	18.6 ms

[†] End-to-end (preprocess+infer+softmax), N=5000.

V. Results

A. E0: Baseline Calibration

Both paths establish zero-load baselines (Table III). GPU latency (10.6 ms mean; 10.9 ms P_{99}) reflects end-to-end pipeline time; CPU latency (14.5 ms mean) reflects pure ONNX Runtime execution. GPU floating-point variance produces non-zero δ_{mean} but zero STER; CPU is arithmetically deterministic. Top-1 accuracy is 1.5% on both paths (label-space mismatch; accuracy is a control variable only).

B. E1: CPU Stress and Timing Divergence

Both paths maintain STER= 0.0000 at all CPU load levels. The GPU path is invariant (≈10.6 ms); the CPU path reaches 73.7 ms at 75% load (5.1× baseline). TensorRT FP16 executes on the GPU, isolated from saturated CPU cores; ONNX Runtime FP32 competes for CPU time slices and cache bandwidth.

C. E2–E4: Isolation Stressors (GPU Path Only)

Memory pressure (25–90% LPDDR5 fill), GPU co-tenancy (0–4 concurrent TensorRT workers), and network I/O (BLE 0–6 + WiFi) all produce STER= 0.0000 with δ_{mean} indistinguishable from E0. GPU DMA isolation and QoS bandwidth reservation prevent all stressor classes from perturbing outputs or latency (+88% interrupt rate at BLE= 4 had no effect).

D. E5: Combined Realistic Load

Under simultaneous CPU 75%, memory 50% fill, 4 BLE connections + fio disk write, the GPU path maintains STER= 0.0000 and latency <11 ms (mean 10.6 ms), 9.8× lower than the CPU path. The CPU path maintains STER= 0.0000 but reaches 104.0 ms mean, 165.1 ms P_{99}, exceeding the 10 Hz budget by 65% (Fig. 2).

E. E6: CPU Contention Characterization

STER= 0.0000 and $\delta_{\text{mean}} = 0.0000$ across all 30,000 inferences; `CPUExecutionProvider` uses deterministic ARM NEON Single Instruction Multiple Data (SIMD) operations. $P_{99} = 165.1$ ms under combined load means 1% of inferences exceed the clinical budget by 65%. The GPU path maintains <11 ms under identical load (9.8× lower), consistent with timing isolation as a property of GPU-accelerated execution stacks under the tested conditions.

TABLE IV. E1 CPU Stress: Both Paths (10 Trials × 500 Inferences Per Level)

CPU Load	GPU STER	GPU Lat.	CPU STER	CPU Lat.
25%	0.0000	≈10.6 ms	0.0000	33.4 ms
50%	0.0000	≈10.6 ms	0.0000	57.8 ms
75%	0.0000	≈10.6 ms	0.0000	73.7 ms
100%	0.0000	≈10.6 ms	0.0000	70.9 ms

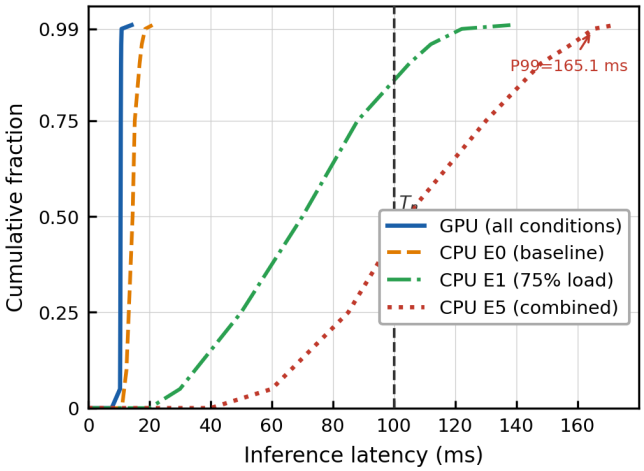

Fig. 2. Empirical cumulative distribution function (CDF) of inference latency for GPU and CPU paths under baseline (E0), CPU stress (E1 75%), and combined load (E5). GPU $P_{99} = 10.9$ ms across all conditions. CPU path exceeds $T_n = 100$ ms at the 75th percentile under E5 ($P_{99} = 165.1$ ms); undetected by accuracy-based validation (Δacc $< 1.0\%$).

TABLE V. E6 CPU-Only Contention: ONNX Runtime FP32 (10 Trials × 500 Inferences)

Condition	STER	δ_{mean}	Lat. mean	Lat. SD	P_{99}
Zero load	0.0000	0.0000	14.5 ms	0.2 ms	18.6 ms
CPU 25%	0.0000	0.0000	33.4 ms	1.8 ms	111.4 ms
CPU 50%	0.0000	0.0000	57.8 ms	3.2 ms	115.8 ms
CPU 75%	0.0000	0.0000	73.7 ms	4.1 ms	122.1 ms
CPU 100%	0.0000	0.0000	70.9 ms	3.8 ms	117.0 ms
Combined†	**0.0000**	**0.0000**	**104.0 ms**	**5.7 ms**	**165.1 ms**

†Exceeds $T_n = 100$ ms budget by 65%.

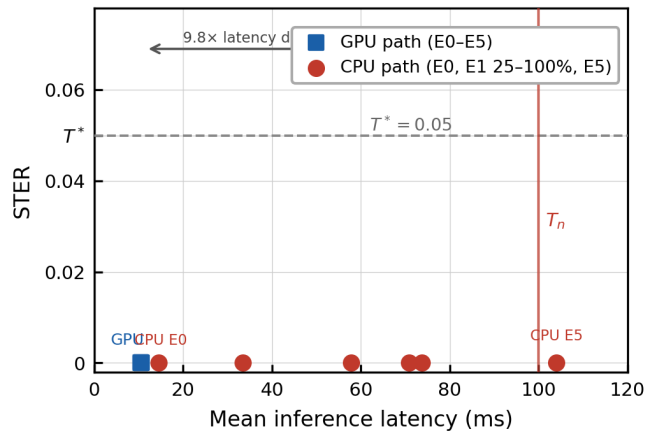

Fig. 3. STER vs. mean latency for all experimental conditions. Both GPU and CPU paths maintain STER$= 0$ (below $T^* = 0.05$) regardless of latency. GPU clusters at 10.6 ms across all conditions; CPU spans 14.5–104.0 ms, breaching $T_n = 100$ ms while STER remains zero. Output stability and timing reliability are structurally independent properties.

VI. Discussion

The finding is not that timing degrades under load, but that existing accuracy-based validation cannot detect this degradation, as demonstrated empirically here.

A. Controlled Design as Methodological Contribution

The same-hardware design controls for all hardware-level confounds; CPU-path timing degradation of 7.2× (14.5 to 104 ms mean) and the 9.8× GPU/CPU contrast under combined load reflect the combined effect of execution path architecture, runtime (TensorRT vs. ONNX), and precision (FP16 vs. FP32). Further validation including controlled ablation studies isolating individual factors, and testing against regulated device workflows, is required before 510(k) adoption.

B. Timing as the Dominant Failure Mode

STER$= 0.0000$ on both paths is not a null result: it is the necessary condition that makes the timing finding interpretable. Without confirmed output stability, an observed CPU-path latency increase could reflect output-level degradation (slower inference due to numerical instability) rather than pure scheduling pressure. STER$= 0$ rules this out: scheduling pressure delays computation without perturbing arithmetic, making timing degradation the sole identifiable safety-relevant divergence (Fig. 3). This motivates timing characterization as an independent, complementary pre-market verification property, one that accuracy-based validation is structurally unable to provide.

C. Operationalizing Pre-Market Robustness Requirements

A manufacturer using accuracy-only testing would clear both paths as equivalent; only joint condition (3) distinguishes them: Δacc$<1.0\%$ on both paths while the CPU path breaches the 10 Hz budget by 65%. This limitation is structural: accuracy evaluates the argmax of a single inference output and carries no information about the temporal spacing of successive outputs, yet FDA-2024-D-4488 requires robustness assessment under foreseeable conditions of use. A device operating at 10 Hz produces one inference activation every 100 ms; if that activation requires 165 ms, the clinical loop stalls regardless of whether the output is numerically correct. FDA-2024-D-4488 requires robustness assessment under foreseeable conditions of use [1]; the joint STER + latency condition operationalizes this requirement at the inference layer. Neither STER alone nor latency alone is sufficient: STER$= 0$ on the CPU path confirms output correctness while the latency violation confirms deployment infeasibility, and only their conjunction exposes the failure.

D. STER as Output Stability Certification

STER certifies that the full output distribution $\sigma(\mathbf{y})$ remains within T^* under all tested conditions. Accuracy (top-1 class) cannot provide this certification as it does not observe subthreshold distribution shifts. STER$= 0$ on both paths isolates timing degradation as the sole safety-relevant divergence. This isolation is the methodological contribution: without a confirmed output-stability metric, a reviewer cannot distinguish between two failure hypotheses: (1) the model output degrades under contention, or (2) only timing degrades. STER$= 0$ rules out hypothesis (1) across all tested conditions, making the timing failure the clean and attributable result. The $T^* = 0.05$ threshold is a proposed value requiring clinical validation for specific device classes; tighter thresholds would not change the finding since $\delta_{\max}$ never exceeded 0.025 across any of the 107,500 inference activations. Appropriate thresholds should be established

through risk analysis per ISO 14971 [9]. Future work should establish clinically validated thresholds for specific device classes, and extend the protocol to non-deterministic runtimes where $\delta > 0$ may appear under contention even without argmax changes.

E. Limitations

Results are specific to MobileNetV2 at 10 Hz on Jetson Orin Nano Super under JetPack 6. $T^* = 0.05$ is a proposed threshold requiring clinical validation for specific device classes. The GPU and CPU paths differ in runtime (TensorRT vs. ONNX), precision (FP16 vs. FP32), and measurement scope (end-to-end vs. runtime-only); the 9.8× latency difference reflects the combined effect of these factors and cannot be attributed solely to architectural isolation. The STER$= 0$ finding is precision-independent and holds across both execution stacks. This study does not evaluate preprocessing latency, real-time operating system (OS) scheduling, multi-model pipelines, or a complete regulated clinical workflow.

VII. Conclusion

Output correctness and timing reliability are independent safety properties of edge AI inference pipelines, as demonstrated by comparing two complete execution stacks (TensorRT GPU FP16 vs. ONNX CPU FP32) on the same hardware platform. Both paths maintain STER$= 0$ across all conditions; the GPU path maintains latency below 11 ms while the CPU path degrades 7.2× to 104 ms mean ($P_{99} = 165$ ms, 9.8× higher), breaching the 10 Hz budget by 65%. Accuracy-based pre-market testing would pass both paths without detecting this timing failure. Joint STER + latency verification is proposed as a candidate method for operationalizing FDA-2024-D-4488 robustness requirements at the inference layer, consistent with IEC 62304 [8] and ISO 14971 [9], and subject to regulatory review and clinical validation for specific device classes.

Data Availability

Experimental scripts, raw results, and analysis code are publicly available at https://github.com/akulswami/p2-edge-ai-samd.

References

[1] FDA, "Artificial Intelligence-Enabled Device Software Functions: Lifecycle Management and Marketing Submission Recommendations. Draft Guidance," Docket FDA-2024-D-4488, Jan. 2025.

[2] J. Hao, P. Subedi, L. Ramaswamy, and I. K. Kim, "Reaching for the Sky: Maximizing Deep Learning Inference Throughput on Edge Devices with AI Multi-Tenancy," *ACM Trans. Internet Technol.*, vol. 23, no. 1, pp. 1–33, Feb. 2023.

[3] J. J. Martín, A. Castillo, J. García, and F. J. Cazorla, "Performance Isolation for Inference Processes in Edge GPU Systems," arXiv:2601.07600, Jan. 2026.

[4] S. P. Baller, A. Jindal, M. Chadha, and M. Gerndt, "DeepEdgeBench: Benchmarking Deep Neural Networks on Edge Devices," in *Proc. IEEE IC2E*, 2021, pp. 20–30.

[5] M. Tobiasz et al., "Edge Devices Inference Performance Comparison," arXiv:2306.12093, Jun. 2023.

[6] B. A. Becker, "Increasing Safety of Neural Networks in Medical Devices," in *Proc. SAFECOMP Workshops*, LNCS vol. 11699, Springer, 2019, pp. 91–101.

[7] R. Wilhelm et al., "The Worst-Case Execution Time Problem—Overview of Methods and Survey of Tools," *ACM Trans. Embed. Comput. Syst.*, vol. 7, no. 3, pp. 36:1–36:53, Apr. 2008.

[8] IEC 62304:2006+AMD1:2015, "Medical device software: Software life cycle processes," IEC, Geneva, 2015.

[9] ISO 14971:2019, "Medical devices: Application of risk management to medical devices," ISO, Geneva, 2019.

[10] C. I. King, "stress-ng: Tool to Load and Stress a Computer System," GitHub, 2023. [Online]. Available: https://github.com/ColinIanKing/stress-ng

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520.

[12] NVIDIA Corporation, "TensorRT Developer Guide," NVIDIA Developer Documentation, 2024. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/

[13] Microsoft Corporation, "ONNX Runtime: Cross-Platform Inference Accelerator," GitHub, 2024. [Online]. Available: https://github.com/microsoft/onnxruntime

[14] B. Lee et al., "Early Recalls and Clinical Validation Gaps in Artificial Intelligence-Enabled Medical Devices," *JAMA Health Forum*, vol. 6, no. 8, p. e253172, Aug. 2025.